

GalaxEye Classifier

Workflow

API

|

Service

|

Pre Processing

|

Model/Prediction

|

Result(Class and Prob of all class)

|

Post Processing(prob $\rightarrow$ class)

|

Decision Layer

|

Store Result in SQLite with metadata(img, result, model confident, need_review, class prob, etc.)

Design note

First trained a model using CNN in colab. and store the weight. Recreate the model layers and use the trained weight for inference.

Training

Train a small CNN model with FNN to predict class of the image.

Load images using PIL, do a simple pre process like scale the pixel values, also make the model dimension suitable for pytorch. Split data into train, val, test in 80:10:10 ratio. Because we have small set of weights use max weights for trianing.

Create a small cnn layers and fnn layer to predict o/p. Use cross entropy as loss function for multi classification and adam as optimizer.

Train the model with early stopping to stop the model from overfitting using val data. Monitor both model loss and model accuracy. In this task we need to build a system around the model that need a extra decision layer to process the output. So while training we need to focus to train a model with focus on model loss than model accuracy. Model loss will helps to understand how confident the model is? Like the output logit of each class differ significantly if model loss is less. Meanwhile model accuracy only focus on if answer is correct or not. So for early stopping use model loss. Also store the best model weight and use for inference.

Why simple cnn train from scratch, instead of pre trianed/Multi Modal LLM?

We already get enough data to train a small CNN model and also accuracy is not the core of this task. Also while using pre trained model, there are other problem like data distribution, it mostly trained on other images and for diff dimension images. Also there is a lot of layers and weights in it, so our 100+ sample may be not enough to re train it. Also we can use a small MLLM but we could not verify its output properly, we can only verify model reasoning or its own predicted score that is not a proper evidence. Also it is computationally heavy and not much cpu friendly. One more advantage is the size of the model is relatively very less compared to other two.

Inference System

There should be five parts:

- API - Accept image(single tile).
- Pre process - Pre process the image for model.
- Predict - Create model and predict o/p → convert to probability → select best class.
- Db - Store result in db(Schema, query etc.)
- Service - Connect all these component. We pass img from api to service. In service img → pre process → model → post process → decision layer → db → return response to api

How tile flow through the system?

api → pre processing → model predict → post processing → decision layer → db → response

Input Validation and Preprocessing

- Images only
- Only accept: 64 * 64 dim png images
- Convert to [B, C, H, W]
- Scale b/w 0 to 1
- Use torchvision transform ToTensor for pre processing

Model/Prediction and Post Processing

Recreate model architecture used in training, and load the stored weights to the model. Convert the model for inference, using `model.eval` and inference mode. Then load model on startup.

Convert model logits into softmax. Select high prob class and convert it into its corresponding label.

How to handle Model output? - Decision Layer

- Find high probability class and assign it into the correct label.
- First thing to do is to check model output probability, predicted class have prob greater than our threshold, also make sure output class have a significant output prob diff b/w all other class.
- If any of this false, store it with result and metadata with `need_review = True`,
- Otherwise store the result and metadata with `need_review = False`,

Using class probability we can know how confident model is and using margin calculation we get to know about the uncertainty in the model. So these two can be considered as good measure to detect model failures.

Threshold

For threshold use the training details. Observe samples in validation data and find threshold from it.

How probabilities look like when sample result is false? Use those as threshold here.

DB

After model prediction and post process we need to store the result + metadata for analyst quireies. There are a lot of db we can use for it. But for a PoC and for the purpose on offline, we can consider `sqlite`(python inbuilt one). So we don't need to configure another server for it like `postgres`. Work in a file in our project folder. Easy to manage.

DB Design

Schema:

`id`: int - primary key of the row

`img_name`: str - name of img to understand the given image.

`img_path`: str - path of img in our system.

`predicted_cls`: str - model result.

pred_cls_prob: numeric: prob of predicted cls.

prob: object: store the predicted prob of all class.

need_reivew: bool- if result need review true otherwise false.

review_reason: str - reason for review.

review_status: str: - if a prediction need review and we reviewed it.

model_version: str - version of model used.

Here for now, we are not going to store actual images, so img_path is empty for every row.

need_review values are: 0,1

reivew_reason values: Null, low_confidence, low_margin, both

review_status values: Null, reviewed, pending

Analyst Queries

After the model prediction and result are stored in quuries, analyst need to query from it and get a analysis from it. Here we need to decide what the analyst quuries? Based on our schema we can predict some analyst questions like

- No of images in each class
- Low confidence prediction
- Fetch result need verification
- How many images model can predict properly
- Images processed in a day or b/w a time period
- Get the proportion of images in each class
- Filter prediction by confidence/margin

etc.. there is a lot of scope for it. We implement some of the basic queries from analyst with /query endpoint.